



Norme di Progetto

Gruppo Coderius

`coderius01@gmail.com`

Versione 1.1.0

Tabella di versionamento

Versione	Data	Autore	Verificatore	Descrizione
1.1.0	2026/09/02	Filippo Zonta Rocha		Aggiornamento migliorativo, aggiunta sezione 2.4, 2.5, 2.6
1.0.1	2026/07/03	Edis Hodja	Filippo Zonta Rocha	Aggiornamento contenuti della descrizione relativa ai ruoli
1.0.0	2026/06/23	Edis Hodja		Approvazione del documento
0.10.1	2026/06/13	Giovanni Bronte	Edis Hodja	Modifiche alle tabelle delle metriche
0.10.0	2026/05/28	Alberto Canavese	Giovanni Bronte	Aggiunta della seguente sezione: 5.3.2.2
0.9.0	2026/05/15	Edis Hodja	Giovanni Bronte	Aggiunta delle seguenti sezioni: 5.1, 5.2, 5.3
0.8.2	2026/05/15	Leonardo Lorenzin	Filippo Zonta Rocha	Aggiornamento contenuti e correzioni refusi
0.8.1	2026/05/13	Filippo Zonta Rocha	Leonardo Lorenzin	Aggiornamento contenuti e correzioni refusi
0.8.0	2026/05/13	Edis Hodja	Filippo Zonta Rocha	Aggiunta delle seguenti sezioni: 4.3, 4.4, 4.5
0.7.0	2026/05/11	Edis Hodja	Filippo Zonta Rocha	Aggiunta delle seguenti sezioni: 4.1, 4.2
0.6.0	2026/05/08	Edis Hodja	Filippo Zonta Rocha	Aggiunta della sezione 3.5

0.5.0	2026/05/05	Edis Hodja	Filippo Zonta Rocha	Aggiornamento delle seguenti sezioni: 2.2.2, 2.2.3, 2.2.4, 2.3.2, 2.3.5, 3.2.3, 3.3
0.4.0	2026/05/04	Giovanni Bronte	Alberto Canavese	Aggiunta della sezione 3.4
0.3.0	2026/04/30	Giovanni Bronte	Alberto Canavese	Aggiunta della sezione 3.2
0.2.0	2026/04/27	Lorenzin Leonardo	Alberto Canavese	Aggiunta delle sezioni 3.1 e 3.3
0.1.0	2026/04/23	Lorenzin Leonardo	Alberto Canavese	Prima stesura del documento

Indice

1. Introduzione	1
1.1. Scopo del Documento	1
1.2. Scopo del prodotto	1
1.3. Glossario	1
1.4. Riferimenti	1
1.4.1. Riferimenti Normativi	1
1.4.2. Riferimenti Informativi	2
2. Processi Primari	2
2.1. Introduzione ai processi primari	2
2.2. Processo di Fornitura	2
2.2.1. Attività previste	3
2.2.2. Documentazione coinvolta	3
2.2.3. Gestione delle comunicazioni	5
2.2.3.1. Comunicazioni con la proponente _G	5
2.2.3.2. Comunicazioni interne	5
2.2.4. Strumenti a supporto	6
2.2.4.1. Strumenti di comunicazione con la proponente _G	6
2.2.4.2. Strumenti per il gruppo	6
2.3. Processo di Sviluppo	6
2.3.1. Attività previste	6
2.3.2. Analisi dei Requisiti _G	7
2.3.2.1. Nomenclatura Casi D'uso	7
2.3.2.2. Nomenclatura Requisiti _G	8
2.3.3. Documentazione coinvolta	8
2.3.4. Strumenti a supporto	9
2.3.4.1. Strumenti per la proponente _G	9
2.3.4.2. Strumenti per il gruppo	9
2.3.5. Rapporti con la proponente _G	9
2.3.6. Rapporti con il committente	10
2.4. Progettazione del prodotto	10
2.5. Codifica del prodotto	10
2.6. Tecnologie selezionate	11
3. Processi di Supporto	12
3.1. Documentazione	12
3.1.1. Tipologie di documenti	13
3.1.1.1. Verballi	13
3.1.1.2. Diario di bordo	14
3.1.1.3. Denominazione dei documenti	14
3.1.2. Strumenti per la documentazione	14
3.1.3. Produzione	15

3.1.4. Manutenzione	15
3.2. Gestione della configurazione	16
3.2.1. Attività previste	16
3.2.2. Identificazione della configurazione	16
3.2.3. Controllo della configurazione	16
3.2.4. Rendicontazione dello stato della configurazione	17
3.2.5. Valutazione della configurazione	17
3.2.6. Strumenti utilizzati	17
3.3. Garanzia della qualità	18
3.3.1. Attività previste	18
3.3.2. Linee guida	18
3.3.3. Ruoli	19
3.4. Verifica	19
3.4.1. Attività previste	19
3.4.2. Implementazione del processo	19
3.4.3. Verifica	20
3.4.3.1. Analisi statica	20
3.4.3.2. Analisi dinamica	20
3.4.3.2.1. Test di unità _G	20
3.4.3.2.2. Test di integrità	21
3.4.3.2.3. Test di regressione	21
3.4.3.2.4. Test di sistema _G	21
3.4.3.2.5. Test di accettazione	21
3.5. Validazione	21
3.5.1. Attività previste	21
4. Processi Organizzativi	22
4.1. Introduzione ai processi organizzativi	22
4.2. Gestione del processo	22
4.2.1. Attività previste	22
4.2.2. Metodologia di sviluppo	23
4.2.3. Organizzazione del team e ruoli	23
4.2.4. Pianificazione delle milestone _G	24
4.2.5. Verbali	24
4.2.6. Gestione degli sprint _G	25
4.2.7. Monitoraggio dell'avanzamento	25
4.3. Gestione dell'infrastruttura	26
4.3.1. Attività previste	26
4.3.2. Repository e hosting	26
4.3.3. Infrastruttura di sviluppo	27
4.3.4. Gestione dell'ambiente di lavoro	27
4.4. Ottimizzazione del processo	27
4.4.1. Attività previste	28

4.4.2. Retrospective e revisione interna	28
4.4.3. Gestione delle criticità	28
4.4.4. Aggiornamento delle procedure	29
4.5. Formazione	29
4.5.1. Attività previste	29
4.5.2. Condivisione delle conoscenze	29
4.5.3. Studio delle tecnologie	30
4.5.4. Supporto tra membri del gruppo	30
5. Metriche della qualità	30
5.1. Introduzione alle metriche della qualità	30
5.1.1. Classificazione	30
5.1.2. Obiettivi per la Qualità di Prodotto	31
5.1.2.1. Funzionalità	31
5.1.2.2. Affidabilità	31
5.1.2.3. Efficienza	31
5.1.2.4. Usabilità	31
5.1.2.5. Manutenibilità	32
5.1.2.6. Portabilità	32
5.1.3. Nomenclatura e Struttura	32
5.2. Metriche di Qualità del Processo	33
5.2.1. Processi Primari	33
5.2.2. Processi di Supporto	36
5.2.3. Processi Organizzativi	38
5.2.4. Affidabilità	40
5.2.5. Usabilità	42
5.2.6. Efficienza	43
5.2.7. Manutenibilità	43

Elenco delle Tabelle

Tabella 1	Descrizione del documento «Norme di progetto»	3
Tabella 2	Descrizione del documento «Piano di progetto»	3
Tabella 3	Descrizione del documento «Piano di qualifica»	4
Tabella 4	Descrizione del documento «Analisi dei requisiti_G»	4
Tabella 5	Descrizione del documento «Glossario»	4
Tabella 6	Descrizione del documento «Verbali»	4
Tabella 7	Descrizione del documento «Lettera di candidatura»	4
Tabella 8	Descrizione del documento «Valutazione dei Capitolati»	5
Tabella 9	Descrizione del documento «Dichiarazione degli Impegni»	5

1. Introduzione

1.1. Scopo del Documento

Il presente documento ha l'obiettivo di definire il *Way of Working*, ovvero l'insieme di regole, convenzioni e standard che il gruppo si impegna a rispettare durante l'intero ciclo di vita del progetto.

Questo documento verrà redatto contestualmente all'avanzamento delle attività; potrà quindi subire variazioni, integrazioni o rimozioni sulla base delle conoscenze apprese durante il corso del progetto.

1.2. Scopo del prodotto

Il prodotto finale è un'applicazione *web-based* sviluppata per automatizzare la verifica della conformità alla norma **EN 18031**, ossia lo standard tecnico europeo che stabilisce i requisiti di sicurezza dei dispositivi radio wireless. Lo scopo dell'applicazione è quello di guidare l'utente attraverso un'analisi strutturata basata su **decision tree**, che consentono di percorrere i flussi logici della normativa in modo sistematico. Questo approccio non solo velocizza l'intero processo di valutazione, ma ne aumenta l'affidabilità riducendo il rischio di sviste manuali. Infine, il sistema è in grado di produrre automaticamente i report e la documentazione tecnica necessaria per attestare l'esito dell'analisi svolta.

1.3. Glossario

All'interno delle [Norme di Progetto](#), così come negli altri documenti formali, i termini che trovano una definizione specifica nel relativo documento [Glossario](#) verranno contrassegnati da una lettera «G» maiuscola a pedice (es. Termine_G).

Questa convenzione permette al lettore di individuare immediatamente i vocaboli che possiedono un significato particolare nel contesto del progetto, invitandolo a consultarne la definizione per evitare ambiguità riguardo al linguaggio tecnico utilizzato e garantire così una migliore comprensione dei contenuti.

1.4. Riferimenti

1.4.1. Riferimenti Normativi

I documenti elencati di seguito rappresentano i vincoli diretti e le norme da rispettare per lo sviluppo del progetto:

- [Capitolato C1 — Automated EN18031_G Compliance Verification](#) (ultimo accesso: 2026-05-22)
- [Regolamento del Progetto Didattico](#) (ultimo accesso: 2026-04-30)
- [Standard ISO/IEC 12207:2017](#) (ultimo accesso: 2026-04-27)
- **Standard EN 18031** (Consultato tramite copia fornita dalla proponente_G)

1.4.2. Riferimenti Informativi

- Dispense del corso di Ingegneria del Software 2025/2026:
 - ▶ [T01 - Introduzione](#) (ultimo accesso: 2026-04-23)
 - ▶ [T02 - I processi di ciclo di vita del software](#) (ultimo accesso: 2026-04-27)
 - ▶ [T03 - Modelli di sviluppo software](#) (ultimo accesso: 2026-04-27)
 - ▶ [T04 - Gestione di progetto](#) (ultimo accesso: 2026-04-30)
 - ▶ [T05 - Analisi dei requisiti_G](#) (ultimo accesso: 2026-05-11)
 - ▶ [T06 - Progettazione software](#) (ultimo accesso: 2026-05-13)
 - ▶ [T07 - Qualità di prodotto](#) (ultimo accesso: 2026-06-13)
 - ▶ [T08 - Qualità di processo](#) (ultimo accesso: 2026-06-13)
 - ▶ [T09 - Verifica e validazione: introduzione](#) (ultimo accesso: 2026-05-15)
 - ▶ [T10 - Verifica e validazione: analisi statica](#) (ultimo accesso: 2026-05-15)
 - ▶ [T11 - Verifica e validazione: analisi dinamica](#) (ultimo accesso: 2026-05-15)
- [Standard ISO/IEC 12207:1995](#) (ultimo accesso: 2026-04-30)
- [Documentazione ufficiale di Typst_G](#) (ultimo accesso: 2026-04-23)
- [Guida a Git e GitHub](#) (ultimo accesso: 2026-04-23)

2. Processi Primari

2.1. Introduzione ai processi primari

Per realizzare un software funzionante e di qualità, il prodotto finale deve poter essere **robusto** e **mantenibile** nel tempo.

A tal fine è necessario adottare un modello di lavoro basato su **procedure chiare**, capaci di guidare ogni attività in modo ordinato.

Seguendo lo standard **ISO/IEC 12207**, i processi primari principali si suddividono in:

- **Fornitura**: riguarda tutto ciò che serve per gestire il rapporto con la proponente_G esterna;
- **Sviluppo**: riguarda le attività pratiche per costruire il software vero e proprio.

2.2. Processo di Fornitura

Il processo di fornitura definisce le procedure che il gruppo **Coderius**, in qualità di fornitore, si impegna a seguire per gestire il rapporto con la proponente_G **BlueWind S.r.l.**

durante tutto il ciclo di vita del progetto al fine di garantire la corretta fruizione del prodotto finale.

2.2.1. Attività previste

Di seguito le principali attività comprese nel processo di fornitura:

- **Analisi del capitolato:** esame critico del documento di proposta per identificare requisiti e vincoli tecnici.
- **Studio di fattibilità:** valutazione della sostenibilità del progetto in termini di risorse, tempi e competenze tecniche.
- **Pianificazione:** organizzazione temporale delle attività e delle risorse, formalizzata nel **Piano di Progetto**.
- **Monitoraggio e Controllo:** verifica costante dell'avanzamento dei lavori e della qualità dei prodotti intermedi, anche a seguito di eventuali feedback da parte della proponente.
- **Rilascio e Accettazione:** procedure di consegna del software e della documentazione, con successiva validazione da parte della proponente.

2.2.2. Documentazione coinvolta

Le attività legate al processo di fornitura si concretizzano nella redazione e nel continuo aggiornamento dei seguenti documenti:

Norme di progetto	
Redattore	Amministratore
Tipo di documento	Interno
Scopo	Definisce le regole operative, le convenzioni e gli standard tecnici che il gruppo deve adottare per garantire l'uniformità dei processi.

Tabella 1: Descrizione del documento «Norme di progetto»

Piano di progetto	
Redattore	Responsabile
Tipo di documento	Esterno
Scopo	Stabilisce la pianificazione delle attività, la gestione delle risorse, l'analisi dei rischi e la ripartizione dei costi.

Tabella 2: Descrizione del documento «Piano di progetto»

Piano di Qualifica	
Redattore	Amministratore
Tipo di documento	Esterno
Scopo	Specifica gli obiettivi di qualità, le metriche di misurazione e i test necessari a validare il software e i documenti prodotti.

Tabella 3: Descrizione del documento «Piano di qualifica»

Analisi dei Requisiti _G	
Redattore	Analista
Tipo di documento	Esterno
Scopo	Formalizza le necessità della proponente _G in una lista strutturata di requisiti _G funzionali, qualitativi e di vincolo.

Tabella 4: Descrizione del documento «Analisi dei requisiti_G»

Glossario	
Redattore	Team (assegnazione dinamica)
Tipo di documento	Esterno
Scopo	Raccoglie la terminologia tecnica e i termini potenzialmente ambigui per assicurare una comprensione univoca di tutta la documentazione.

Tabella 5: Descrizione del documento «Glossario»

Verbali	
Redattore	Team (assegnazione dinamica)
Tipo di documento	Esterno/Interno
Scopo	Documenti di coordinamento che registrano l'esito delle riunioni, l'assegnazione dei compiti interni e dei ruoli assegnati. Nel caso dei verbali esterni, hanno anche lo scopo di tracciare decisioni, risposte e vincoli concordati.

Tabella 6: Descrizione del documento «Verbali»

Lettera di candidatura	
Redattore	Responsabile
Tipo di documento	Esterno
Scopo	Documento per la richiesta formale da parte del gruppo di poter partecipare al capitolato scelto tramite la validazione dei capitolati.

Tabella 7: Descrizione del documento «Lettera di candidatura»

Valutazione dei Capitolati	
Redattore	Analista
Tipo di documento	Esterno
Scopo	Descrive l'analisi comparativa effettuata sulle diverse proposte per giustificare la scelta del capitolato finale.

Tabella 8: Descrizione del documento «Valutazione dei Capitolati»

Dichiarazione degli Impegni	
Redattore	Responsabile
Tipo di documento	Esterno
Scopo	Documento formale in cui il gruppo presenta una stima dei costi del progetto e delle ore che verranno impiegate per il suo completamento. Vengono inoltre dichiarate le responsabilità assunte e la data di consegna prevista.

Tabella 9: Descrizione del documento «Dichiarazione degli Impegni»

2.2.3. Gestione delle comunicazioni

Per mantenere una comunicazione trasparente e tracciabile, a seconda della tipologia di incontro, il gruppo adotta le seguenti regole:

2.2.3.1. Comunicazioni con la proponente_G

Riguardano tutti i contatti ufficiali tra il gruppo **Coderius** e **BlueWind S.r.l.**

- **Incontri (meeting):** ogni incontro con la proponente_G deve essere documentato in un **verbale esterno**, denominato secondo il formato *VerbaleEsterno-AAAA-MM-GG*, che riporti le decisioni prese e i chiarimenti ottenuti.
- **Comunicazioni formali:** per l'invio di documenti e comunicazioni ufficiali si utilizza esclusivamente la posta elettronica.
- **Canali rapidi:** per chiarimenti immediati e coordinamento si utilizzano strumenti di messaggistica istantanea (Telegram), attraverso i quali la proponente_G condivide risorse tecniche di riferimento.

2.2.3.2. Comunicazioni interne

Riguardano il coordinamento tra i membri del gruppo **Coderius**.

- **Incontri (meeting):** ogni incontro tra i componenti del gruppo deve essere documentato in un **verbale interno**, denominato secondo il formato *VerbaleInterno-AAAA-MM-GG*, che riporti le decisioni prese e i compiti da svolgere.
- **Canali rapidi:** per il coordinamento e gli aggiornamenti si utilizzano strumenti di messaggistica istantanea (WhatsApp, Discord).

2.2.4. Strumenti a supporto

Per il corretto svolgimento delle attività del processo di fornitura, il gruppo adotta i seguenti strumenti, distinguendoli in base all'ambito di utilizzo:

2.2.4.1. Strumenti di comunicazione con la proponente_G

- **Gmail:** utilizzato per le comunicazioni formali e la convocazione dei meeting.
- **Telegram:** utilizzato per chiarimenti immediati.
- **Zoom:** utilizzato per lo svolgimento dei meeting programmati per tempo.

2.2.4.2. Strumenti per il gruppo

- **GitHub:** utilizzato per il versionamento e la conservazione di tutto il materiale prodotto (codice e documentazione), garantendo la tracciabilità delle versioni e delle modifiche.
- **Discord / WhatsApp:** utilizzati per il coordinamento quotidiano e gli incontri tra i componenti del gruppo.
- **Google Docs:** utilizzato per la collaborazione in tempo reale, in particolare per attività di brainstorming e redazione condivisa (es. definizione delle domande per gli incontri con la proponente_G).

2.3. Processo di Sviluppo

Il processo di sviluppo comprende l'insieme delle attività tecniche e metodologiche necessarie alla realizzazione del prodotto software, a cominciare dall'[Analisi dei Requisiti_G](#) fino alla consegna e alla verifica finale. L'obiettivo principale è garantire un ciclo di vita del software controllato, dove ogni fase contribuisce alla creazione di un sistema **robusto, sicuro, mantenibile nel tempo e conforme** alle specifiche della proponente_G.

2.3.1. Attività previste

Di seguito vengono riportate le principali attività comprese nel processo di sviluppo:

- **Analisi dei Requisiti_G:** processo di individuazione, classificazione e formalizzazione delle necessità della proponente_G e dei vincoli normativi, comprensivo della definizione dei casi d'uso del sistema.
- **Progettazione dell'architettura:** definizione della struttura ad alto livello del sistema, con l'individuazione dei componenti principali e le loro interazioni.
- **Progettazione di dettaglio:** definizione specifica dei singoli componenti e delle logiche interne, propedeutica alla scrittura del codice.
- **Codifica:** implementazione del codice sorgente seguendo le convenzioni e gli standard di stile stabiliti dal gruppo in fase di progettazione.

- **Test e Integrazione:** attività volta a verificar il corretto funzionamento dei singoli componenti e la loro successiva combinazione, assicurando la coerenza e l'interoperabilità dell'intero sistema realizzato.

2.3.2. Analisi dei Requisiti_G

Questa fase mira a definire con precisione cosa il software debba fare. Il gruppo adotta le seguenti procedure:

- **Analisi delle fonti:** studio del capitolato e della documentazione messa a disposizione dalla proponente_G al fine di estrarre i vincoli tecnici e normativi.
- **Individuazione dei casi d'uso (UC):** descrizione delle interazioni tra gli attori e il software tramite diagrammi UML e scenari (principali o alternativi).
- **Classificazione dei requisiti_G:** ogni requisito viene catalogato come funzionale, di qualità, di vincolo o prestazionale.
- **Attribuzione della priorità:** a ogni requisito viene assegnata una priorità (obbligatorio, desiderabile o opzionale) e un identificativo univoco secondo una specifica nomenclatura.
- **Tracciamento:** associazione costante tra requisiti_G e casi d'uso per garantire che ogni necessità richiesta dal capitolato sia soddisfatta e che ogni caso d'uso non risulti superfluo.

2.3.2.1. Nomenclatura Casi D'uso

I casi d'uso sono identificati mediante la seguente convenzione:

UC-[IndiceRadice].[IndiceAnnidato]: Titolo

Dove:

- **UC:** acronimo di **Use Case_G** (caso d'uso);
- **IndiceRadice:** numero progressivo che identifica un caso d'uso principale;
- **IndiceAnnidato:** numero crescente che indica la sottosezione di un particolare use case_G, può essere ulteriormente annidato per rappresentare livelli successivi;
- **Titolo:** nome assegnato a ogni use case_G per differenziarlo dagli altri e riassumerne lo scopo, migliorandone la leggibilità.

La combinazione tra **IndiceRadice** e **IndiceAnnidato** costituisce un identificativo univoco e non duplicabile.

Esempio:

UC-4: Aggiunta nuovo dispositivo manuale;

UC-4.1: Inserimento nome dispositivo;

UC-4.2: Inserimento sistema operativo del dispositivo.

2.3.2.2. Nomenclatura Requisiti_G

Per i requisiti_G si utilizza la seguente nomenclatura:

R[Tipologia]-[Priorità][Codice]

La dicitura ha questi significati:

- **[Tipologia]**: indica il tipo di requisito:
 - **F (Funzionale)**: descrive una funzionalità del sistema.
 - **Q (Qualitativo)**: descrive una caratteristica qualitativa del sistema.
 - **V (Vincolo)**: descrive un vincolo logico o normativo.
- **[Priorità]**: indica l'ordine di importanza dell'implementazione:
 - **Ob (Obbligatorio)**: requisito richiesto esplicitamente dalla proponente_G o necessario al funzionamento del sistema.
 - **D (Desiderabile)**: requisito che aumenta il valore del prodotto, ma non è indispensabile.
 - **Op (Opzionale)**: requisito da realizzare solo se il resto del lavoro è stato completato.
- **[Codice]**: numero progressivo intero che, insieme alla tipologia, costituisce un identificativo univoco dei requisiti_G.

Esempio:

RF-Ob01: Il sistema deve permettere l'inserimento di un nuovo dispositivo all'interno della piattaforma.

RF-D03: Il sistema deve permettere l'annullamento della procedura di eliminazione di un dispositivo durante la fase di richiesta di conferma.

RQ-Ob07: La gestione del ciclo di vita del progetto e l'organizzazione delle attività del team devono adottare un approccio Agile_G, garantendo iterazioni regolari e flessibilità.

2.3.3. Documentazione coinvolta

All'inizio del nostro progetto didattico ci siamo resi conto che il lavoro non poteva procedere per tentativi o per una spartizione approssimativa dei compiti, soprattutto considerando il tempo più ridotto a disposizione per la suddivisione dei gruppi del II lotto.

Le attività del processo di sviluppo sono supportate dalla documentazione definita in precedenza, ovvero [Analisi dei Requisiti_G](#), [Piano di Progetto](#), [Piano di Qualifica e Norme di Progetto](#), integrata con specifiche riguardanti la codifica e le procedure di verifica.

2.3.4. Strumenti a supporto

Per lo sviluppo, il gruppo utilizza strumenti mirati a garantire qualità, tracciabilità e collaborazione:

- **Linguaggio e Ambiente:** come linguaggio principale per lo sviluppo delle componenti software e delle logiche della normativa viene utilizzato Python 3.x;
- **Versionamento:** Git e GitHub per la gestione del repository, dei branch, dei commit, delle issue e delle pull request;
- **Automazione:** utilizzo di pipeline CI (Continuous Integration) tramite GitHub Actions per la generazione automatica e la successiva pubblicazione della documentazione di progetto.

2.3.4.1. Strumenti per la proponente_G

- **Gmail:** utilizzato per comunicazioni formali, convocazione dei meeting;
- **Telegram:** utilizzato per chiarimenti immediati;
- **Zoom:** utilizzato per lo svolgimento dei meeting.

2.3.4.2. Strumenti per il gruppo

- **Discord / WhatsApp:** utilizzati per il coordinamento quotidiano e incontri tra i componenti del gruppo;
- **Draw.io/PlantUML:** utilizzati per la realizzazione dei diagrammi UML che sono essenziali per l'analisi dei requisiti_G, in particolar modo per l'interpretazione dei casi d'uso.

2.3.5. Rapporti con la proponente_G

La proponente_G **BlueWind S.r.l.** mette a disposizione del gruppo strumenti e momenti di confronto per agevolare lo sviluppo del progetto:

- **Canali di comunicazione:** un canale Telegram per comunicazioni rapide e informali, e l'invio di email tramite l'indirizzo del gruppo per chiarimenti tecnici e condivisione di documentazione ufficiale;
- **Incontri remoti:** la possibilità di organizzare meeting sulla piattaforma Zoom, a seguito dei quali il gruppo redigerà un **Verbale Esterno** per tracciare le decisioni prese;
- **Materiale di supporto:** la fornitura della documentazione tecnica relativa alla norma **EN 18031** e indicazioni sulle logiche dei **decision tree**_G da implementare.

Il gruppo **Coderius** si impegna a recepire i feedback ricevuti e a presentare un prodotto conforme ai requisiti_G concordati entro le scadenze prefissate.

2.3.6. Rapporti con il committente

Il committente mette a disposizione strumenti di supporto e momenti di valutazione per monitorare la qualità dei processi e dei prodotti:

- **Ricevimenti e comunicazioni:** possibilità di richiedere incontri o inviare e-mail istituzionali per chiarire dubbi metodologici o riguardanti la gestione del progetto;
- **Diari di Bordo:** incontri periodici in cui il gruppo espone l'avanzamento dei lavori, le difficoltà riscontrate e le soluzioni adottate.

Il gruppo si impegna a partecipare alle milestone [G](#) di avanzamento previste:

- **RTB (Requirements and Technology Baseline):** momento di revisione che prevede la consegna della documentazione e la presentazione di un **Proof of Concept** (PoC) per dimostrare la fattibilità tecnica della soluzione scelta;
- **PB (Product Baseline):** fase finale che prevede la consegna della documentazione tecnica completa e la presentazione dell'applicazione funzionante per la verifica della conformità allo standard EN 18031.

2.4. Progettazione del prodotto

L'attività di progettazione del prodotto è svolta dai progettisti del gruppo, che si occupano di definire l'architettura e la struttura del software, garantendo che il sistema sia modulare, scalabile e facilmente manutenibile. La progettazione avviene in due fasi principali:

- **Scelta delle tecnologie:** definizione degli strumenti, dei linguaggi di programmazione e dei framework da utilizzare per lo sviluppo del software, tenendo conto delle esigenze del progetto e delle competenze del team.
- **Progettazione dell'architettura:** definizione dei componenti principali del sistema delineando la struttura logica e man mano definendo nel dettaglio le singole unità software, le loro responsabilità e le interazioni tra di esse.

Si tratta di un processo collaborativo che collide con le idee e le competenze dei membri del team, con l'obiettivo di creare un prodotto finale coerente.

Il processo si raggiunge attraverso l'adozione di design pattern e di best practice di ingegneria del software, che permettono di ottenere un'architettura solida scalabile e facilmente manutenibile.

Per mantenere la coerenza progettuale vengono definiti i test che saranno successivamente utilizzati per verificare il corretto funzionamento del sistema e la loro conformità ai requisiti [G](#).

2.5. Codifica del prodotto

L'attività di codifica del prodotto è svolta dai programmatori del gruppo, che si occupano di implementare le funzionalità definite nella fase di progettazione. La codifica avviene

seguendo le linee guida stabilite nelle **Norme di Progetto**, che includono convenzioni di nomenclatura, standard di stile e best practice di programmazione.

- **Convenzione della nomenclatura:**

- Classi: i nomi delle classi seguono la convenzione PascalCase sia nel front end che nel back end.
- Metodi, funzioni e variabili: i nomi dei metodi, delle funzioni e delle variabili seguono la convenzione CamelCase nel front end e Snakecase nel back end.
- Costanti: Uppersnake.
- Nomi File: CamelCase nel front end e Snakecase nel backend^G.

- **Formattazione e linting:**

- Per garantire uniformità di stile il codice è sottoposto a controlli automatici di formattazione e stile tramite strumenti di linting integrati nella pipeline continuous integration: ogni Pull Request viene verificata e il merge è bloccato in caso di violazione delle convenzioni stabilite. Un file di configurazione condiviso (.editorconfig) uniforma le regole editoriali di base.

- **Best practice:**

- Logica di dominio separata da framework/infrastruttura.
- Lint e test locali obbligatori prima della PR.
- Approvazione di un membro del team prima del merge.

- **Lingua e commenti:**

- Identificatori di codice (classi, funzioni, variabili) sempre in inglese.
- Commenti, messaggi utente e stringhe di errore rivolte all'utente sempre in italiano.
- Nessun docstring/JSDoc: codice autoesplicativo tramite naming, commento solo per motivare scelte non ovvie.
- Convenzione di tracciabilità: i commenti citano l'identificativo del caso d'uso.
- La stessa convenzione (commenti italiani) si estende ai workflow continuous integration.

2.6. Tecnologie selezionate

- Python: utilizzato come linguaggio di programmazione ad alto livello orientato agli oggetti per la realizzazione della logica di backend^G. È stato scelto per la sua versatilità e l'ampia disponibilità di risorse.
- React: utilizzato come framework per la realizzazione della logica di frontend^G.

- Flask: utilizzato come micro-framework per la realizzazione della logica di backend. Ritenuto più adatto alle esigenze del progetto.
- Docker: utilizzato per la containerizzazione dell'applicazione, garantendo stabilità tra gli ambienti di sviluppo e produzione.
- Git: utilizzato come sistema di controllo di versione distribuito per tracciare le modifiche al codice sorgente e facilitare la collaborazione tra i membri del team.
- GitHub: utilizzato come piattaforma di hosting per il repository Git, consentendo la gestione delle pull request, delle issue e della documentazione/codice del progetto.
- GitHub Actions: utilizzato per l'automazione dei processi di build, test e deployment, integrando la pipeline di continuous integration (CI).
- Visual Studio Code: IDE utilizzato come ambiente di sviluppo condiviso.

Per un approfondimento sulle tecnologie scelte, si rimanda al documento [Specifica Tecnica](#).

3. Processi di Supporto

La seguente sezione descrive i processi di supporto secondo lo standard **ISO/IEC 12207**, che hanno lo scopo di garantire una gestione efficace, il controllo e la qualità delle attività del ciclo di vita del progetto.

Nello specifico, vengono analizzati i seguenti processi di supporto:

- **Documentazione;**
- **Gestione delle configurazioni;**
- **Garanzia della qualità;**
- **Verifica;**
- **Validazione.**

3.1. Documentazione

Il processo di documentazione ha lo scopo di raccogliere, organizzare e gestire l'insieme dei documenti prodotti durante l'intero ciclo di vita del progetto. Non rappresenta una semplice attività di archiviazione, ma costituisce il fondamento per garantire la trasparenza, la manutenibilità e la continuità del lavoro tra i membri del gruppo.

Ogni attività, decisione tecnica o organizzativa deve essere formalizzata per permettere:

- **Tracciabilità:** ricostruire l'evoluzione del progetto e le motivazioni dietro ogni scelta architeturale o normativa.

- **Qualità:** assicurare che ogni documento prodotto rispetti gli standard definiti e sia sottoposto a verifica prima del rilascio.
- **Comunicazione:** facilitare lo scambio di informazioni tra i componenti del team e verso la proponente^g e il committente.

3.1.1. Tipologie di documenti

Il gruppo ha definito dei template specifici per garantire l'uniformità e la professionalità della documentazione. I documenti che dispongono di una struttura predefinita sono:

- **Verbale interno:** resoconto delle riunioni di coordinamento del team.
- **Verbale esterno:** documentazione degli incontri con la proponente^g o il committente.
- **Diario di bordo:** documento informativo periodico sullo stato di avanzamento del progetto.

3.1.1.1. Verbali

I verbali del gruppo Coderius seguono una struttura fissa per facilitare la reperibilità delle informazioni:

- **Frontespizio:** include il logo del gruppo, i dati di contatto (coderius01@gmail.com), la tipologia di documento, i riferimenti temporali (data, ora di inizio e fine), i riferimenti logistici (luogo o canale di comunicazione) della riunione e la versione più recente reperita dalla tabella di versionamento.
- **Tabella di versionamento:** riporta lo storico delle modifiche, indicando versione, data, autori, verificatori e descrizione dell'aggiornamento.
- **Indice:** elenco numerato delle sezioni per una navigazione rapida del documento.
- **Partecipanti:** lista dei membri del team presenti all'incontro.
- **Ordine del Giorno:** elenco sintetico dei punti che si intendono affrontare durante la seduta.
- **Svolgimento della Riunione:** discussione dettagliata di quanto anticipato nella sezione ordine del giorno e altre decisioni prese per l'avanzamento del progetto.
- **Conclusioni e Decisioni prese:** riassunto di quanto è stato discusso durante la riunione, sintetizzato in una tabella.
- **Ordine del giorno prossimo incontro:** anticipazione degli argomenti principali di cui discutere durante il successivo incontro.
- **TODO (Attività da svolgere):** tabella riassuntiva che elenca i compiti assegnati in vista dell'incontro successivo.

Convenzioni di nomenclatura e codifica

Per garantire la tracciabilità tra discussioni e azioni, il gruppo adotta i seguenti formati:

- **Attività (TODO):** identificate dal codice **TD-x.y** (dove x è il numero progressivo del verbale e y l'identificativo dell'attività).
- **Decisioni di riferimento:** collegate alle attività tramite il codice **VI_{GG}-x.y** per i verbali interni (dove G a pedice indica il riferimento al Glossario).
- **Date:** espresse nel formato **AAAA/MM/GG** per uniformità con il registro delle modifiche e gli standard internazionali.

3.1.1.2. Diario di bordo

Questa tipologia di documento ha la funzione di monitorare periodicamente lo stato di avanzamento del lavoro da parte del gruppo con finalità prettamente informativa e didattica. La struttura dei Diari di Bordo si articola in tre punti chiave:

- **Risultati raggiunti:** elenco dettagliato delle attività portate a termine nel periodo di riferimento.
- **Attività future:** pianificazione dei task previsti per il prossimo sprint_G.
- **Difficoltà riscontrate:** esposizione di eventuali criticità o ostacoli emersi durante lo svolgimento del progetto.

3.1.1.3. Denominazione dei documenti

Al fine di garantire un ordinamento cronologico intuitivo e una gestione efficiente dei file nel repository, il gruppo ha stabilito una nomenclatura standard basata sul formato **NomeDocumento-AAAA-MM-GG**. Nello specifico:

- **Verbali interni:** VerbaleInterno-AAAA-MM-GG.
- **Verbali esterni:** VerbaleEsterno-AAAA-MM-GG.
- **Diari di bordo:** DiarioDiBordo-AAAA-MM-GG.

3.1.2. Strumenti per la documentazione

Per la redazione e la gestione della documentazione, il gruppo ha deciso di utilizzare i seguenti strumenti:

- **Typst_G:** è il principale linguaggio di markup utilizzato per l'intera documentazione di progetto. A differenza di sistemi più datati come LaTeX, Typst_G offre una compilazione istantanea e una sintassi più moderna e leggibile. Questo permette al gruppo di mantenere un alto standard qualitativo dei documenti, garantendo al contempo una maggiore velocità nella revisione dei contenuti e nella gestione dei template personalizzati.

- **GitHub:** rappresenta il fulcro del sistema di conservazione e controllo dei documenti. L'utilizzo di repository dedicate permette di tracciare ogni singola modifica tramite il versionamento del codice sorgente dei file .typ. Il gruppo sfrutta inoltre le seguenti funzionalità:
 - **Pull Request:** per gestire i processi di verifica e approvazione in modo strutturato prima dell'integrazione definitiva.
 - **GitHub Actions:** per automatizzare la generazione dei PDF a partire dai sorgenti Typst^G, garantendo che l'ultima versione prodotta sia sempre disponibile e correttamente formattata.
 - **Issue e Projects:** per la pianificazione dei task documentali e il monitoraggio delle scadenze.

3.1.3. Produzione

Per la produzione di un documento il nostro gruppo opera nel seguente modo:

- **Creazione Issue e Branch dedicato:** inizialmente viene creata una issue su GitHub dal responsabile assegnato in quello sprint^G. In base all'importanza del documento, viene creato un relativo branch di feature per poterci lavorare senza influenzare il main. Se il documento da creare è invece di minore rilevanza, come un verbale, questo viene inserito nel branch apposito feature/verbali, il quale contiene al suo interno sia i verbali esterni sia quelli interni.
- **Stesura del documento:** il documento viene assegnato a uno o più membri del team che ricoprono il ruolo adatto per la stesura in quello specifico sprint^G.
- **Verifica del documento:** ogni volta che il documento viene aggiornato, quest'ultimo deve essere verificato da chi ricopre il ruolo di verificatore, il quale aggiornerà la tabella di versionamento inserendo il proprio nome e una breve descrizione delle modifiche apportate.
- **Approvazione e Merge:** una volta che il documento è concluso viene aperta una pull request su GitHub dal responsabile, inizialmente verso il branch develop e in seguito verso il main. Dal momento in cui il documento è presente nel branch main viene considerato concluso e, grazie all'automazione di GitHub Actions, il PDF viene generato e pubblicato automaticamente.

3.1.4. Manutenzione

Nel caso in cui i documenti necessitino di modifiche a causa di eventuali errori presenti all'interno del loro contenuto, il gruppo segue i seguenti passaggi:

1. Creare un nuovo branch di lavoro dedicato per la correzione degli errori (es. fix/norme-progetto).
2. Apportare le modifiche necessarie in locale.

3. Integrare la tabella di versionamento dello specifico documento inserendo una nuova riga che spieghi l'intervento effettuato.
4. Effettuare il push sul branch di lavoro.
5. Aprire una Pull Request in modo tale che il documento aggiornato possa essere verificato dal membro del gruppo assegnato a tale attività.
6. Aggiungere il documento revisionato nel branch dedicato a quest'ultimo (es. feature/norme-progetto).

3.2. Gestione della configurazione

Questa sezione ha l'obiettivo di descrivere come il gruppo Coderius gestisce le componenti software e documentali nel progetto, al fine di garantire tracciabilità, coerenza e controllo delle modifiche.

3.2.1. Attività previste

La gestione delle attività previste avviene nel seguente ordine:

- **Identificazione della configurazione;**
- **Controllo della configurazione;**
- **Rendicontazione dello stato della configurazione;**
- **Valutazione della configurazione.**

3.2.2. Identificazione della configurazione

Secondo lo standard **ISO/IEC 12207:1995**, l'attività di identificazione della configurazione consiste nell'individuazione di tutti gli elementi del progetto, includendo sia le componenti software composte da codice sia l'intera parte documentale.

Questa attività viene effettuata durante la fase di progettazione per schematizzare l'architettura del software, la quale verrà in seguito implementata dagli sviluppatori.

3.2.3. Controllo della configurazione

Per poter svolgere al meglio questa attività, il gruppo Coderius adotta i seguenti strumenti:

- **Pull Request:** permette al team di revisionare il lavoro svolto e verificarlo prima di effettuarne il merge nel branch di destinazione (develop o main). Questo processo viene seguito per ridurre al minimo il rischio di errori prima che un documento o codice sia considerato concluso.
- **Issue:** permettono di tracciare, gestire e pianificare le attività del progetto. Vengono aperte alla redazione di un nuovo documento o all'avvio di una funzionalità, in modo da poter assegnare il task ai membri del team preposti per quello sprint. Ogni issue presenta le seguenti caratteristiche:

- È identificata all'interno di GitHub tramite la notazione #NUMERO, dove NUMERO rappresenta un identificativo univoco del repository.
- Descrive l'attività da svolgere o la modifica richiesta.
- Può essere assegnata a uno o più membri del team.
- Viene aggiornata durante l'avanzamento delle attività fino alla sua chiusura definitiva.
- **Project Board:** è una tavola digitale utilizzata per tracciare lo stato delle attività in tempo reale. Le colonne principali utilizzate sono:
 - **Backlog:** attività da pianificare e non ancora avviate.
 - **Ready:** attività pronte per essere sviluppate nello sprint_G corrente.
 - **In Progress:** attività attualmente in fase di sviluppo.
 - **In Review:** attività in fase di verifica e revisione.
 - **Done:** attività completate e integrate nel branch principale.

3.2.4. Rendicontazione dello stato della configurazione

La rendicontazione dello stato della configurazione definisce le regole di versionamento dei file. Il gruppo ha adottato la codifica standard **MAJOR.MINOR.PATCH**:

- **MAJOR:** viene incrementata in caso di rilasci ufficiali del documento.
- **MINOR:** viene incrementata in caso di modifiche sostanziali, come l'aggiunta di nuove sezioni in un documento o di nuove funzionalità nel codice.
- **PATCH:** viene incrementata in caso di modifiche minori, correzioni di refusi o piccoli bug fix ortografici.

3.2.5. Valutazione della configurazione

Questa attività serve a controllare che il software prodotto sia pienamente funzionale e completo rispetto ai requisiti_G individuati. Tale controllo viene garantito grazie a un tracciamento costante dei requisiti_G, formalizzato e aggiornato all'interno del documento [Analisi dei Requisiti_G](#).

3.2.6. Strumenti utilizzati

Gli strumenti per la gestione della configurazione che il gruppo ha deciso di adottare sono i seguenti:

- **GitHub:** principale strumento per creare, modificare e gestire tutti i file legati al progetto. Permette inoltre di definire le milestone_G, assegnare i compiti e tracciare l'avanzamento del lavoro.

- **GitHub Pages:** servizio utilizzato per l'hosting del sito web del gruppo, direttamente connesso al repository principale. Il suo scopo è consentire un accesso facile e rapido alla proponente^G e ai committenti per la consultazione di tutta la documentazione prodotta in formato PDF.

3.3. Garanzia della qualità

Il gruppo implementa il processo di garanzia della qualità (*Quality Assurance*) con l'obiettivo di fornire evidenze concrete e verificabili che il prodotto software, i processi di sviluppo e tutta la documentazione prodotta siano conformi ai requisiti^G definiti, agli standard adottati e alle pianificazioni stabilite.

3.3.1. Attività previste

Il processo di garanzia della qualità prevede le seguenti attività:

- **Implementazione del processo:** definizione e adozione delle metriche e delle procedure di qualità che verranno applicate durante l'intero ciclo di vita del progetto.
- **Controllo di processo:** verifica periodica che le attività svolte dai membri del gruppo siano aderenti alle procedure e agli standard stabiliti.
- **Accertamento del prodotto:** verifica costante che il prodotto software e la relativa documentazione rispettino i requisiti^G specifici e le convenzioni stabilite dal gruppo.
- **Valutazione oggettiva:** la verifica e garanzia della qualità, quando possibile, viene affidata a membri del team che non hanno partecipato attivamente alla stesura del documento o della porzione di codice in questione.

3.3.2. Linee guida

Il processo di garanzia della qualità segue le seguenti linee guida:

- **Prevenzione prima della correzione:** il gruppo privilegia pratiche preventive, come revisioni e verifiche periodiche, rispetto alla correzione a posteriori delle anomalie.
- **Tracciabilità:** modifiche, verifiche e anomalie sono documentate e tracciate tramite gli strumenti adottati dal team.
- **Miglioramento continuo:** procedure e modalità operative vengono periodicamente rivalutate per migliorarne l'efficacia e l'efficienza.
- **Responsabilità condivisa:** la qualità del prodotto e della documentazione è una responsabilità condivisa da tutti i membri del gruppo.
- **Conformità agli standard:** ogni artefatto prodotto deve rispettare le convenzioni, i requisiti^G e gli standard definiti dal gruppo.

3.3.3. Ruoli

Nella seguente tabella sono riportati i ruoli principali ed essenziali per garantire la qualità del prodotto e della documentazione di progetto:

Ruolo	Specifica
Responsabile	Supervisiona le attività di garanzia della qualità e il rispetto degli standard definiti.
Verificatore	Esegue verifiche su documentazione, codice e artefatti prodotti.
Amministratore	Gestisce gli strumenti di automazione e supporto alle verifiche.
Membro del gruppo	Contribuisce al mantenimento della qualità seguendo le procedure e le convenzioni condivise.

3.4. Verifica

Il processo di verifica è tecnico e oggettivo; ha lo scopo di valutare e determinare se il lavoro svolto (codifica e documentazione prodotta) soddisfa i requisiti necessari e concordati con la proponente. L'obiettivo di questo processo è di dimostrare in modo oggettivo la correttezza del prodotto rispetto ai requisiti documentati nel capitolato di progetto e destinati all'utilizzo dell'utenza finale. Questo processo è continuo per la durata del progetto ed è definito nel [Piano di Qualifica](#).

3.4.1. Attività previste

Secondo lo standard **ISO/IEC 12207:1995**, il processo di verifica prevede le seguenti attività:

- **Implementazione del processo:** questa attività consiste unicamente nel predisporre il piano di verifica che effettuerà il controllo del lavoro svolto. Consiste nell'individuare le criticità e i rischi di non conformità del progetto.
- **Verifica:** esecuzione di attività di controllo e revisione su documentazione, codice e altri artefatti presenti, al fine di individuare eventuali non conformità e garantirne la correttezza;

3.4.2. Implementazione del processo

La seguente attività ha come scopo principale l'individuazione delle criticità che possono sorgere durante la realizzazione del progetto. Per tutte le criticità che possono essere riscontrate è stata condotta un'analisi accurata nella sezione 3 del documento [Piano di Progetto](#). Questa analisi include anche: probabilità di occorrenza, gravità della criticità, strategie di mitigazione e di prevenzione.

3.4.3. Verifica

Le attività di verifica vengono svolte tramite il sistema di *Pull Request* adottato dal gruppo su GitHub. Ogni modifica viene sottoposta a revisione prima dell'integrazione nel ramo principale del repository.

Per ciascuna Pull Request viene assegnato uno o più *reviewer*, a seconda di chi ricopre il ruolo di verificatore per lo sprint_G corrente, incaricato/i di controllare la conformità dell'artefatto rispetto ai requisiti_G, alle convenzioni e agli standard definiti.

Le attività di verifica vengono svolte da membri differenti dagli autori degli artefatti analizzati, così da garantire una valutazione quanto più oggettiva possibile.

3.4.3.1. Analisi statica

L'analisi statica viene così definita in quanto non richiede l'esecuzione del programma esaminato. Il suo scopo è verificare la sintassi e correttezza del documento o codice in esame.

Questa analisi può essere effettuata attraverso i seguenti metodi:

- **Walkthrough:** metodo manuale che richiede un'analisi critica dell'intero file. Viene utilizzato principalmente quando si rileva un'anomalia di cui non si conosce l'esatta collocazione o causa. Questo approccio è dispendioso in termini di tempo e di costi; per questo motivo il team punta a utilizzarlo il meno possibile, preferendo verifiche automatizzabili.
- **Inspection:** metodo formalizzato e guidato da una checklist predefinita. Questa checklist ha lo scopo di individuare gli errori ricorrenti e verificare criteri di qualità specifici. Trattandosi di un metodo parzialmente automatizzabile e più strutturato, il gruppo predilige l'uso dell'inspection rispetto al walkthrough.

3.4.3.2. Analisi dinamica

L'analisi dinamica, a differenza di quella statica, richiede l'esecuzione del programma per rilevare errori e anomalie nel comportamento a runtime.

Lo strumento principale di questa tecnica è il *test*, che per essere efficace deve rispettare due proprietà fondamentali: la ripetibilità, ovvero la capacità di produrre i medesimi esiti a parità di condizioni, e l'automatizzabilità, ovvero la possibilità di essere eseguito senza intervento manuale.

Queste proprietà sono essenziali per supportare i test di regressione: se un errore precedentemente risolto dovesse ripresentarsi, l'automazione permette di individuarlo tempestivamente e determinare quale modifica lo abbia reintrodotta.

3.4.3.2.1. Test di unità_G

I test di unità_G verificano una singola unità di codice (funzione, metodo o componente) in isolamento, sostituendo le sue dipendenze con dei fake o dei mock. Nel backend_G vengono

realizzati con *pytest*, nel frontend_G con *Vitest*. I file di test sono collocati accanto al codice sorgente che verificano, seguendo la convenzione `test_*.py` per il backend_G e `*.test.ts/ *.test.tsx` per il frontend_G.

3.4.3.2.2. Test di integrità

I test di integrità verificano la corretta interazione tra più componenti reali del sistema, senza sostituire le dipendenze interne con mock: nel backend_G attraversano rotta HTTP, servizio e repository tramite il test client dell'applicazione; nel frontend_G coinvolgono pagina, store e client API insieme. Adottano gli stessi strumenti e la stessa convenzione di nomenclatura dei test di unità_G.

3.4.3.2.3. Test di regressione

I test di regressione verificano che le modifiche apportate al codice non abbiano introdotto regressioni, ovvero malfunzionamenti in funzionalità precedentemente corrette.

3.4.3.2.4. Test di sistema_G

I test di sistema_G verificano il comportamento del sistema nel suo complesso, se rispettano i requisiti_G funzionali e qualitativi definiti nel capitolato di progetto. Vengono eseguiti in un ambiente di test che simula quello di produzione, con l'obiettivo di individuare eventuali problemi prima del rilascio finale.

3.4.3.2.5. Test di accettazione

I test di accettazione verificano che il sistema soddisfi i requisiti_G funzionali e qualitativi definiti nel capitolato di progetto.

3.5. Validazione

Il processo di validazione rappresenta la fase conclusiva attraverso cui si accerta che il prodotto sviluppato soddisfi i requisiti_G definiti dalla proponente_G e risponda correttamente agli obiettivi del progetto. La validazione ha lo scopo di verificare che il sistema realizzato sia conforme alle esigenze espresse nel capitolato e che le funzionalità implementate risultino adeguate al contesto applicativo finale.

Nel contesto del progetto, il soggetto di riferimento per l'accettazione e la validazione è l'azienda proponente_G, la quale certifica la conformità del software alle necessità operative emerse durante le attività di analisi.

3.5.1. Attività previste

- **Implementazione del processo:** definisce le modalità operative attraverso cui viene accertata la conformità del prodotto rispetto ai requisiti_G specifici del capitolato e della documentazione di progetto.

La validazione viene inoltre supportata dall'utilizzo del caso studio fornito dal proponente_G, così da verificare il comportamento del sistema in un contesto applicativo realistico e coerente con gli obiettivi del progetto. Gli esiti dei test svolti vengono tracciati nel [Piano di Qualifica](#).

- **Validazione:** ha lo scopo di accertare che il sistema sviluppato soddisfi i requisiti funzionali e gli obiettivi definiti nel capitolato di progetto.

Le attività di validazione coinvolgono il ruolo di *Responsabile*, che controlla la correttezza funzionale delle componenti realizzate, con particolare attenzione all'esecuzione dei decision tree, alla gestione dei requisiti EN18031 e alla produzione degli esiti previsti (PASS, FAIL, NOT APPLICABLE).

4. Processi Organizzativi

4.1. Introduzione ai processi organizzativi

I processi organizzativi definiscono l'insieme delle attività e delle procedure adottate dal gruppo per coordinare il lavoro, gestire le risorse e garantire il corretto avanzamento del progetto durante l'intero ciclo di vita del software.

L'obiettivo principale di questi processi è garantire continuità operativa, collaborazione efficace tra i membri del gruppo e capacità di adattamento rispetto alle esigenze emerse durante lo sviluppo del progetto.

Nel contesto del progetto, i processi organizzativi riguardano in particolare:

- **Gestione del Processo;**
- **Gestione dell'Infrastruttura;**
- **Ottimizzazione del Processo;**
- **Formazione.**

4.2. Gestione del processo

La gestione del processo definisce le modalità con cui il gruppo pianifica, organizza e monitora le attività necessarie allo sviluppo del progetto. L'obiettivo principale è garantire una distribuzione efficace del lavoro, il rispetto delle scadenze concordate e il controllo continuo dell'avanzamento delle attività.

4.2.1. Attività previste

Le attività previste per la gestione del processo sono le seguenti:

- **Definizione della metodologia di sviluppo**
- **Organizzazione del team e assegnazione dei ruoli**
- **Pianificazione delle milestone e degli sprint**
- **Gestione operativa delle attività**
- **Monitoraggio dell'avanzamento del progetto**

• Gestione delle comunicazioni e della documentazione organizzativa

4.2.2. Metodologia di sviluppo

Il gruppo adotta un approccio di sviluppo Agile_G, basato su cicli iterativi e incrementali della durata di due settimane ciascuno, denominati sprint_G. Questa scelta nasce dalla natura del progetto, che richiede un confronto frequente con la proponente_G e comporta la necessità di affinare progressivamente l'interpretazione dei requisiti_G e delle soluzioni progettuali.

In particolare, il modello adottato si caratterizza per i seguenti aspetti:

- *Iterazioni brevi e cadenzate*: ogni sprint_G ha una durata fissa di due settimane, al termine delle quali il gruppo valuta i risultati ottenuti e pianifica le attività successive.
- *Sviluppo incrementale*: ogni sprint_G produce un incremento funzionante del sistema, riducendo il rischio di accumulare lavoro non verificato e rendendo più agevole il controllo della qualità nel tempo.
- *Adattabilità*: la struttura iterativa permette al gruppo di reagire tempestivamente a cambiamenti nei requisiti_G o a criticità emerse durante lo sviluppo, senza compromettere la stabilità complessiva del progetto.

4.2.3. Organizzazione del team e ruoli

Ciascuno dei membri del gruppo ricopre un ruolo che varia a ogni sprint_G in base a un criterio di rotazione equo e distribuito. Questa scelta, oltre a essere un requisito del corso, garantisce che ogni membro acquisisca nel tempo una visione completa del progetto e del processo di sviluppo, riducendo la dipendenza da singole figure e favorendo una collaborazione più consapevole.

Di seguito vengono riportati i vari ruoli e vengono descritte le responsabilità associate a ciascuno di essi:

• Responsabile

Si occupa del monitoraggio delle risorse e gestisce le scadenze ed eventuali rischi. È il ruolo cruciale che autorizza il rilascio dei risultati prodotti, che siano parziali o finali. Il suo impegno è richiesto in maniera costante per l'intera fase di sviluppo progettuale.

• Amministratore

Assicura l'efficienza di procedure, strumenti e tecnologie a supporto del *Way of Working* del team. La sua presenza è costante per l'intero progetto.

• Analista

Si occupa dell'intera analisi e stesura della documentazione relativa all'[Analisi dei Requisiti_G](#), impegnandosi a determinare e soddisfare tutti i requisiti_G richiesti dalla proponente_G, affinché l'intero team riesca a operare in modo efficace.

- **Progettista**

Stabilisce la struttura del software, che costituirà il prodotto richiesto, definendo l'architettura dell'applicazione e la suddivisione tra i componenti Frontend_G e Backend_G.

- **Programmatore**

Rappresenta il ruolo cruciale nella fase di implementazione e codifica, in quanto si occupa di sviluppare il codice sorgente, collaborando con il *Progettista* riguardo alle decisioni stabilite.

- **Verificatore**

Si occupa di accertare il rispetto delle norme di qualità per ogni prodotto intermedio e finale del progetto, dalla documentazione al codice sorgente. Pianifica ed esegue le procedure di test necessarie alla validazione del sistema, segnalando tempestivamente eventuali difetti o non conformità al fine di garantire la correttezza e l'affidabilità del prodotto sviluppato.

Pur essendo assegnati ruoli specifici per ciascuno sprint_G, le attività vengono svolte in modo collaborativo tra i membri del gruppo.

4.2.4. Pianificazione delle milestone_G

Il progetto prevede diverse revisioni di avanzamento, il cui superamento consente al team di proseguire con le attività in modo supervisionato. In particolare, le due principali milestone_G, riportate in ordine cronologico, sono:

- **RTB (Requirements and Technology Baseline)**: fissa i requisiti_G da soddisfare in accordo con la proponente_G nella documentazione dedicata all'[Analisi dei Requisiti_G](#), motiva le tecnologie adottate e ne dimostra l'adeguatezza e l'interoperabilità tramite il *Proof of Concept*.
- **PB (Product Baseline)**: vaglia la maturità della baseline architetturale del software e la sua effettiva realizzazione. Essa prevede l'integrazione dei seguenti elementi:
 - **Specifica Tecnica**: documentazione dedicata all'architettura logica del prodotto, strutturata per componenti, ruoli, connessioni e interazioni.
 - **Minimum Viable Product (MVP)**: approssima da vicino la soluzione finale tramite una versione preliminare ma funzionante, volta a ottenere la conferma ufficiale della conformità del sistema atteso.

4.2.5. Verbalì

Ogni riunione, sia essa interna al gruppo o esterna con la proponente_G, viene verbalizzata e documentata in appositi verbali redatti a rotazione da un membro del team, indipendentemente dal ruolo ricoperto nello sprint_G corrente.

Ogni verbale riporta la data, i partecipanti, gli argomenti discussi, le decisioni prese e le eventuali azioni da intraprendere, costituendo così una traccia formale e consultabile dell'avanzamento del progetto e delle scelte effettuate nel tempo.

4.2.6. Gestione degli sprint_G

Il team adotta un approccio Agile_G per assicurare massima produttività e organizzazione operativa. Il lavoro viene strutturato in *sprint*_G di durata bisettimanale, i quali prevedono una precisa sequenza di coordinamento:

- **Fase iniziale di pianificazione:** il team definisce gli obiettivi da raggiungere nel periodo di riferimento, suddividendoli in unità di lavoro specifiche (*issue*) e categorizzando ciascuna di esse in termini di priorità tramite apposite *label* su GitHub.
- **Stato intermedio:** attraverso i canali di comunicazione stabiliti, i membri del gruppo si confrontano periodicamente per monitorare l'andamento dei task e risolvere eventuali blocchi operativi o criticità.
- **Valutazione e consuntivo:** al termine di ogni ciclo lavorativo viene effettuata un'analisi delle procedure adottate e dei traguardi raggiunti, procedendo alla chiusura delle *issue* completate o alla rendicontazione di quelle rimaste in stato di *work in progress* per la loro riprogrammazione.

4.2.7. Monitoraggio dell'avanzamento

L'avanzamento operativo delle unità di lavoro (*issue*) viene monitorato e gestito attraverso una *Project Board* in stile Kanban. Questo strumento permette di tracciare visivamente il ciclo di vita di ciascuna attività, la quale transita progressivamente attraverso le seguenti fasi:

- **Backlog:** fase iniziale in cui vengono inserite tutte le attività identificate e pianificate per il progetto, in attesa di essere assegnate a uno *sprint*_G specifico.
- **Ready:** raccoglie le attività selezionate e preparate per essere sviluppate durante lo *sprint*_G corrente.
- **In Progress:** indica che l'attività è stata presa in carico dal membro del team assegnato ed è attualmente in fase di sviluppo o redazione.
- **In Review:** fase in cui l'artefatto prodotto viene sottoposto a controllo da parte del *Verificatore* per accertarne la qualità e la conformità agli standard.
- **Done:** fase di chiusura definitiva dell'attività, raggiungibile solo a seguito dell'approvazione del codice o del documento e della successiva integrazione nel ramo principale del repository da parte del *Responsabile*.

4.3. Gestione dell'infrastruttura

La gestione dell'infrastruttura ha lo scopo di definire, realizzare e mantenere nel tempo l'insieme delle infrastrutture, degli ambienti di lavoro e degli strumenti di supporto adottati dal gruppo durante l'intero ciclo di vita del progetto, garantendo che siano sempre adeguati alle esigenze operative e coerenti con il *Way of Working* stabilito.

4.3.1. Attività previste

Il processo di gestione dell'infrastruttura si articola nelle seguenti attività principali:

- **Gestione del repository e dei servizi di hosting;**
- **Configurazione degli ambienti di sviluppo;**
- **Automazione dei processi di integrazione e distribuzione;**
- **Gestione della disponibilità e conservazione dei dati;**
- **Gestione e aggiornamento dell'ambiente di lavoro.**

4.3.2. Repository e hosting

Per la gestione centralizzata del codice e della documentazione viene utilizzato GitHub, adottando un modello ispirato a GitHub Flow che permette di mantenere un workflow semplice, ordinato e coerente con l'approccio Agile^G. Il modello è il seguente:

- **Struttura dei branch:** il ramo principale è rappresentato dal *main*, in cui vengono pubblicate la documentazione e la codifica verificate e definitive. Prima dell'integrazione, ogni modifica viene inserita nel branch di staging *develop* per essere esaminata dal *Verificatore*, che effettua una revisione approfondita. Successivamente, il *Responsabile* autorizza e procede con l'integrazione finale nel ramo principale.
- **Convenzione di denominazione:** per le varie operazioni e attività effettuate viene creato un branch dedicato che segue una struttura concordata dal team:
 - **feature/nome-feature** — per lo sviluppo di nuove funzionalità;
 - **fix/nome-fix** — per la correzione di bug o comportamenti anomali;
- **Revisione e merge:** l'integrazione del lavoro avviene in due fasi distinte, entrambe gestite tramite *Pull Request*. Nella prima fase, il branch di lavoro viene unito a *develop*, previa revisione e approvazione da parte del *Verificatore*. Una volta consolidato il lavoro su *develop*, viene aperta una seconda PR per l'integrazione definitiva su *main*, anch'essa soggetta alla supervisione del *Verificatore* e approvata dal *Responsabile* prima del merge.
- **Pipeline automatizzate:** sono configurate le *GitHub Actions* per automatizzare le operazioni ripetitive, tra cui la compilazione dei documenti Typst^G in PDF e la loro pubblicazione sul sito web del progetto.

4.3.3. Infrastruttura di sviluppo

L'infrastruttura di sviluppo comprende l'insieme degli strumenti, degli ambienti e delle configurazioni adottate dal gruppo per garantire un processo di sviluppo standardizzato, ripetibile e facilmente manutenibile nel tempo. In particolare, il team utilizza i seguenti strumenti:

- **Discord:** Canale vocale di comunicazione principale specialmente utilizzato per gli incontri interni e per la discussione in tempo reale di punti cruciali;
- **WhatsApp:** Canale di comunicazione rapido per messaggistica e brainstorming generale;
- **Zoom:** Strumento di comunicazione sfruttato per la discussione relativa ai *Diari di Bordo* assieme al committente e le riunioni di allineamento con la proponente;
- **Typst:** Linguaggio di markup per la stesura e compilazione dell'intera documentazione prodotta nell'intero ciclo di vita del progetto;
- **Git:** Sistema di versionamento distribuito per codice e documentazione;
- **GitHub:** Piattaforma per la repository di gruppo, issue tracking system e integrazione continua;
- **Google Docs:** Sistema per la stesura di domande e brainstorming su tematiche affrontate durante gli incontri interni.
- **Python:** Linguaggio di programmazione utilizzato per alcune automazioni;

4.3.4. Gestione dell'ambiente di lavoro

Mantenere l'infrastruttura e gli strumenti del progetto aggiornati e funzionanti è una responsabilità continua che accompagna l'intero ciclo di sviluppo. Lo scopo è preservare la continuità operativa e l'affidabilità dell'ambiente, assicurando che tutto resti coerente con gli standard stabiliti dal gruppo.

Gli interventi si distinguono in base alla loro natura:

1. **Interventi ricorrenti:** aggiornamenti e operazioni periodiche legate alla normale evoluzione delle attività;
2. **Interventi straordinari:** modifiche, aggiunte o rimozioni di strumenti, configurazioni o risorse rese necessarie da cambiamenti nei requisiti, criticità impreviste o decisioni organizzative non pianificate.

4.4. Ottimizzazione del processo

L'ottimizzazione del processo ha l'obiettivo di garantire che le modalità di lavoro adottate dal gruppo evolvano nel tempo, adattandosi alle esigenze emergenti e agli standard di qualità definiti. Anziché considerare il *Way of Working* come qualcosa di fisso, il gruppo lo tratta come un elemento vivo: soggetto a osservazione, valutazione e revisione continua lungo l'intero ciclo di vita del progetto.

In questo senso, ogni fase di lavoro rappresenta un'occasione per individuare inefficienze, consolidare le pratiche più efficaci e introdurre correzioni mirate, con l'obiettivo di incrementare progressivamente sia la qualità dei risultati sia la sostenibilità del processo stesso.

4.4.1. Attività previste

L'ottimizzazione del processo si articola in tre attività principali:

- **Definizione delle procedure:** formalizzazione delle modalità operative adottate dal gruppo, con relativa documentazione nelle [Norme di Progetto](#) e nel *Way of Working* stabilito.
- **Monitoraggio e valutazione:** osservazione continua dell'andamento delle attività attraverso retrospettive periodiche, analisi delle criticità emerse e valutazione delle metriche associate al processo.
- **Revisione e miglioramento:** identificazione delle aree di inefficienza e introduzione di correzioni mirate, aggiornando la documentazione ufficiale ogni qualvolta le procedure vengano modificate.

4.4.2. Retrospettive e revisione interna

Al termine di ogni sprint_G il gruppo dedica un momento strutturato alla riflessione collettiva, finalizzato a valutare l'andamento del periodo appena concluso. In questa sede vengono analizzati gli obiettivi raggiunti, le difficoltà incontrate e le dinamiche di collaborazione emerse, con l'intento di trarre indicazioni concrete per lo sprint_G successivo.

Sebbene la conduzione di questi incontri mantenga un carattere flessibile per favorire il confronto aperto, la struttura segue una cadenza ricorrente che garantisce la partecipazione attiva di tutti i membri. Gli esiti della discussione vengono sintetizzati all'interno del relativo verbale.

4.4.3. Gestione delle criticità

Quando si presenta una problematica che rischia di compromettere il regolare svolgimento delle attività, il gruppo segue un iter strutturato articolato nelle seguenti fasi:

1. **Identificazione:** la criticità viene rilevata e descritta in modo chiaro, specificandone il contesto e le attività coinvolte.
2. **Analisi:** si individuano le cause alla base del problema e si valuta l'impatto sulle tempistiche, sulle risorse e sulla qualità del progetto.
3. **Risposta:** in base alla natura e all'urgenza, il gruppo definisce le azioni correttive più appropriate, che possono includere la redistribuzione dei compiti, la revisione delle stime, l'aggiornamento degli strumenti o la modifica delle procedure coinvolte.
4. **Tracciamento:** ogni criticità rilevante viene registrata e monitorata fino alla sua risoluzione, al fine di prevenire il ripresentarsi di situazioni analoghe.

4.4.4. Aggiornamento delle procedure

Le norme e le procedure adottate dal gruppo non sono considerate definitive. Infatti, possono essere riviste e aggiornate in qualsiasi momento del progetto, qualora emerga la necessità di adeguarle a nuove esigenze operative o di correggere aspetti risultati inefficaci nella pratica.

A tal fine, la documentazione ufficiale viene costantemente revisionata e aggiornata per risultare costantemente valida e affidabile in termini di procedure seguite dal gruppo.

4.5. Formazione

Il gruppo riconosce la formazione come una componente attiva del progetto, rifiutando l'idea che si tratti di un'attività puramente accessoria. Acquisire una solida padronanza degli strumenti, dei linguaggi e delle pratiche concordate è una condizione necessaria per operare in modo autonomo ed efficace; per questo motivo, tale processo viene gestito con il medesimo rigore riservato alle fasi di sviluppo e documentazione.

4.5.1. Attività previste

Il processo di formazione si articola in tre attività principali:

- **Studio individuale delle tecnologie:** ogni membro è responsabile dell'acquisizione delle competenze necessarie allo svolgimento dei propri compiti, facendo riferimento alla documentazione ufficiale e alle risorse condivise dal gruppo.
- **Condivisione delle conoscenze:** le competenze acquisite vengono messe a disposizione del team attraverso momenti di confronto, documentazione interna e supporto diretto tra membri.
- **Supporto reciproco:** nei casi in cui un membro incontri difficoltà tecniche o operative, il gruppo interviene attivamente per favorire il superamento del blocco, evitando che le criticità individuali rallentino l'avanzamento collettivo.

4.5.2. Condivisione delle conoscenze

La condivisione delle conoscenze avviene in modo informale ma strutturato: quando un componente del team acquisisce competenze rilevanti per il progetto, è tenuto a renderle accessibili al resto del gruppo. Questo flusso informativo si appoggia alle piattaforme di comunicazione stabilite, come WhatsApp per le notifiche rapide e Discord per sessioni di allineamento vocale o di condivisione dello schermo.

L'obiettivo principale non è la produzione di documentazione formativa esaustiva, bensì l'abbattimento della soglia d'ingresso operativo per chi si trova ad affrontare per la prima volta uno strumento o una tecnologia già appresi da altri membri.

4.5.3. Studio delle tecnologie

Ciascun membro è responsabile dello studio autonomo delle tecnologie assegnate al proprio ambito di lavoro. Come riferimento principale viene utilizzata la documentazione ufficiale dei singoli strumenti, integrata da risorse aggiuntive individuate in autonomia:

- [Typst_G Documentazione Ufficiale](#);
- [Python Documentazione Ufficiale](#); (Riferimento alla versione: 3.14.5)
- [React Documentazione ufficiale](#);
- [Flask Framework](#).

Il percorso di apprendimento non segue una struttura rigida: ogni membro organizza lo studio in base ai propri tempi e alle scadenze del progetto, segnalando eventuali difficoltà al resto del gruppo qualora queste possano impattare sulle attività pianificate.

4.5.4. Supporto tra membri del gruppo

Quando un membro del gruppo si trova in difficoltà, sia in fase di apprendimento sia durante lo svolgimento delle attività, gli altri intervengono per fornire supporto diretto. Questo avviene principalmente attraverso le riunioni interne; tuttavia, in caso di blocchi critici che ostacolano il proseguimento delle attività, vengono organizzate sessioni di lavoro condivise in cui uno o più membri si rendono disponibili per un incontro straordinario, così da far fronte insieme alle difficoltà riscontrate.

Il supporto reciproco non è considerato un'eccezione, bensì parte integrante del *Way of Working* del gruppo: distribuire le competenze in modo omogeneo riduce le dipendenze dai singoli e aumenta la resilienza del team nel suo complesso.

5. Metriche della qualità

5.1. Introduzione alle metriche della qualità

Al fine di garantire un approccio quantitativo alla qualità, il gruppo adotta un sistema di misurazione basato su metriche. Queste rappresentano i criteri oggettivi attraverso i quali viene valutata l'aderenza dei processi e dei prodotti agli standard prefissati.

Le metriche definite in questo documento trovano applicazione e tracciamento operativo all'interno del [Piano di Qualifica](#).

5.1.1. Classificazione

Le metriche sono suddivise in due categorie principali, ispirate allo standard ISO/IEC 9126:

1. **Metriche di Qualità di Processo (MPC)**: indicatori utilizzati per monitorare l'andamento delle attività di progetto, l'efficienza del team e la gestione delle risorse (es. costi, tempi, stabilità dei requisiti_G).
2. **Metriche di Qualità di Prodotto (MPD)**: indicatori focalizzati sulle caratteristiche degli artefatti prodotti, siano essi documentali o software.

5.1.2. Obiettivi per la Qualità di Prodotto

Il gruppo adotta il modello **ISO/IEC 9126** per definire gli obiettivi di qualità, declinando le sei caratteristiche principali in sotto-attributi misurabili.

5.1.2.1. Funzionalità

Valuta la capacità del software di soddisfare le necessità definite dal proponente^g.

- **Adeguatezza:** capacità di svolgere compiti specifici in linea con gli scopi del progetto.
- **Accuratezza:** precisione nel fornire risultati conformi ai requisiti^g definiti.
- **Interoperabilità:** abilità del sistema di scambiare informazioni con componenti esterni.
- **Sicurezza:** grado di protezione dell'integrità dei dati contro accessi non autorizzati.
- **Conformità:** rispetto degli standard di dominio e delle specifiche funzionali.

5.1.2.2. Affidabilità

Misura la stabilità operativa del prodotto in condizioni d'uso standard.

- **Maturità:** capacità di prevenire malfunzionamenti derivanti da errori logici.
- **Tolleranza agli errori:** resilienza del sistema di fronte a input errati o anomalie operative.
- **Recuperabilità:** efficacia delle procedure di ripristino post-guasto.
- **Aderenza:** rispetto di regole di progettazione finalizzate alla continuità del servizio.

5.1.2.3. Efficienza

Analizza il rapporto tra le prestazioni del sistema e le risorse computazionali consumate.

- **Comportamento temporale:** velocità di risposta del sistema alle richieste dell'utente.
- **Utilizzo delle risorse:** gestione ottimizzata di memoria, CPU e banda di rete.
- **Conformità:** rispetto dei vincoli architetturali e di performance.

5.1.2.4. Usabilità

Indaga la facilità con cui l'utente finale interagisce con il prodotto.

- **Comprensibilità:** immediatezza concettuale delle interfacce.
- **Apprendibilità:** curva di apprendimento necessaria per padroneggiare le funzionalità.
- **Operabilità:** controllo fluido e intuitivo dei flussi di lavoro.

- **Attrattività:** qualità estetica e gratificazione dell'esperienza utente.
- **Conformità:** aderenza alle linee guida di usabilità stabilite nelle NdP.

5.1.2.5. Manutenibilità

Descrive la facilità di evoluzione del sistema nel tempo.

- **Analizzabilità:** facilità con cui è possibile diagnosticare la causa di un difetto.
- **Modificabilità:** agilità nell'integrare nuove feature o aggiornare quelle esistenti.
- **Stabilità:** resistenza del software all'introduzione di effetti collaterali a seguito di modifiche.
- **Testabilità:** efficacia delle procedure di verifica nel validare le modifiche apportate.

5.1.2.6. Portabilità

Valuta la flessibilità del software in ambienti eterogenei.

- **Adattabilità:** possibilità di configurazione in contesti operativi differenti.
- **Installabilità:** semplicità della procedura di deploy del software.
- **Conformità:** rispetto di standard di cross-platform.
- **Sostituibilità:** facilità di sostituzione del sistema nel caso di migrazioni tecnologiche.

5.1.3. Nomenclatura e Struttura

Per garantire l'univocità e la reperibilità, ogni metrica_G deve essere definita secondo il seguente formato:

M[TIPO]-[ID]

- **M:** prefisso costante per «Metrica_G»;
- **[TIPO]:** identificatore della categoria (**PC** per Processo, **PD** per Prodotto);
- **[ID]:** numero progressivo a due cifre (es. 01, 02).

Per ogni metrica_G, il gruppo deve definire:

- **Nome e Identificativo;**
- **Descrizione:** scopo della misurazione;
- **Formula di calcolo:** metodo matematico per ottenere il valore;
- **Soglia di Accettabilità:** valore minimo per considerare il requisito soddisfatto;
- **Soglia di Ottimalità:** valore target per un prodotto di eccellenza.

5.2. Metriche di Qualità del Processo

5.2.1. Processi Primari

I processi primari comprendono tutte le attività legate direttamente al ciclo di vita del prodotto software. Per valutarne l'andamento, l'efficienza e la conformità agli obiettivi prefissati, vengono utilizzate metriche quantitative in grado di monitorare l'avanzamento dei lavori, l'allocazione delle risorse e la qualità di ciò che viene prodotto.

L'analisi di questi indicatori consente di individuare eventuali scostamenti e applicare correzioni mirate per garantire il rispetto dei vincoli di progetto.

I range accettabili e desiderabili per ciascuna metrica_G sono definiti nel documento [Piano di Qualifica](#).

Le metriche di seguito presentate definiscono formule, leggende e criteri di misurazione comuni a tutti i progetti.

Metrica _G	MPC-01 - Planned Value (PV)
Descrizione	Valore del lavoro pianificato da completare entro una determinata data.
Formula	$PV = BAC * \% \text{ lavoro pianificato}$
Soglia di Accettabilità	da definire
Soglia di Ottimalità	da definire

Metrica _G	MPC-02 - Earned Value (EV)
Descrizione	Valore del lavoro effettivamente completato in conformità con il budget previsto.
Formula	$EV = BAC * \% \text{ lavoro completato}$
Soglia di Accettabilità	da definire
Soglia di Ottimalità	da definire

Metrica_G	MPC-03 - Actual Cost (AC)
Descrizione	Costo totale effettivamente sostenuto fino alla data corrente.
Formula	$AC = \sum \text{costi sostenuti}$
Soglia di Accettabilità	da definire
Soglia di Ottimalità	da definire

Metrica_G	MPC-04 - Schedule Performance Index (SPI)
Descrizione	Indica l'efficienza temporale. Se < 1, il progetto è in ritardo.
Formula	$SPI = \frac{EV}{PV}$
Soglia di Accettabilità	da definire
Soglia di Ottimalità	da definire

Metrica_G	MPC-05 - Cost Performance Index (CPI)
Descrizione	Indica l'efficienza economica. Se < 1, il progetto è fuori budget.
Formula	$CPI = \frac{EV}{AC}$
Soglia di Accettabilità	da definire
Soglia di Ottimalità	da definire

Metrica_G	MPC-06 - Estimate at Completion (EAC)
Descrizione	Proiezione del costo totale finale basata sulla performance attuale.
Formula	$EAC = \frac{BAC}{CPI}$
Soglia di Accettabilità	da definire
Soglia di Ottimalità	da definire

Metrica_G	MPC-07 - Estimate to Complete (ETC)
Descrizione	Costo stimato necessario per completare il lavoro rimanente.
Formula	$ETC = EAC - AC$
Soglia di Accettabilità	da definire
Soglia di Ottimalità	da definire

Metrica_G	MPC-08 - Requirements Stability Index (RSI)
Descrizione	Misura la stabilità dei requisiti _G nel tempo, valutando l'impatto di modifiche, cancellazioni o aggiunte rispetto al set iniziale.
Formula	$RSI = \frac{NRI - (NC + NRC + NRA)}{NRI}$ Legenda dei parametri: • NRI: Numero Requisiti_G Iniziali • NC: Numero di Cambiamenti • NRC: Numero di Requisiti_G Cancellati • NRA: Numero di Requisiti_G Aggiunti
Soglia di Accettabilità	da definire
Soglia di Ottimalità	da definire

5.2.2. Processi di Supporto

I processi di supporto raccolgono tutte le attività che servono a tenere sotto controllo il ciclo di sviluppo, garantendone tracciabilità e affidabilità nel tempo. Il loro scopo principale è seguire l'andamento dei lavori così da individuare tempestivamente eventuali scostamenti rispetto a quanto pianificato, e poterli correggere prima che diventino critici.

Metrica_G	MPC-09 - Indice di leggibilità Gulpease
Descrizione	Valuta la leggibilità di un testo in lingua italiana. Un indice basso indica un testo troppo complesso per il target di riferimento.
Formula	$IG = 89 + \frac{300 * NF - 10 * NL}{NP}$ Legenda: • NF: Numero di Frasi • NL: Numero di Lettere • NP: Numero di Parole
Soglia di Accettabilità	da definire
Soglia di Ottimalità	da definire

Metrica_G	MPC-10 - Accuratezza Ortografica
Descrizione	Rileva la densità di errori linguistici nei documenti. Il calcolo viene normalizzato su 1000 parole per facilitare il confronto tra documenti di diversa lunghezza.
Formula	$CO = 1000 * \left(\frac{NE}{NP} \right)$ Legenda: • NE: Numero di Errori ortografici • NP: Numero di Parole
Soglia di Accettabilità	da definire
Soglia di Ottimalità	da definire

Metrica_G	MPC-11 - Test Pass Rate
Descrizione	Indica la percentuale di test automatizzati che hanno prodotto un esito positivo rispetto al totale dei test previsti.
Formula	$TSR = \left(\frac{NTS}{NTT} \right) * 100$ Legenda: • NTS: Numero di Test Superati • NTT: Numero Totale dei Test eseguiti
Soglia di Accettabilità	da definire
Soglia di Ottimalità	da definire

Metrica_G	MPC-12 - Code Coverage
Descrizione	Indica la percentuale di linee di codice sorgente effettivamente eseguite durante l'esecuzione della suite di test automatizzati.
Formula	$CC = \left(\frac{NLT}{NLTOT} \right) * 100$ Legenda: • NLT: Numero di linee di codice testate • NLTOT: Numero di linee di codice totali
Soglia di Accettabilità	da definire
Soglia di Ottimalità	da definire

Metrica_G	MPC-13 - Quality Metrics Satisfied
Descrizione	Rappresenta la percentuale complessiva delle metriche di qualità che rientrano nei range accettabili definiti nel Piano di Qualifica.
Formula	$QMS = \left(\frac{NMS}{NMT} \right) * 100$ Legenda: • NMS: Numero di Metriche Soddisfatte • NMT: Numero di Metriche Totali monitorate
Soglia di Accettabilità	da definire
Soglia di Ottimalità	da definire

5.2.3. Processi Organizzativi

Questi processi riguardano la dimensione organizzativa del gruppo: dalla definizione degli standard interni alla gestione della qualità, dallo sviluppo delle competenze al miglioramento continuo. Le metriche associate misurano conformità ed efficacia dei processi di governance, con l'obiettivo di garantire la sostenibilità e la maturità del modello di sviluppo nel tempo.

Metrica_G	MPC-14 - Time Efficiency
Descrizione	Indica il rapporto tra il tempo pianificato e quello effettivamente consumato. Un valore inferiore al 100% segnala che il team sta impiegando più tempo del previsto per le attività.
Formula	$TE = \left(\frac{OPC}{OEC} \right) * 100$ Legenda: • OPC: Ore Previste Cumulative (pianificate) • OEC: Ore Effettive Cumulative (impiegate realmente)
Soglia di Accettabilità	da definire
Soglia di Ottimalità	da definire

Metrica_G	MPD-01 - Requisiti_G Obbligatori Soddisfatti
Descrizione	Indica la percentuale di requisiti _G obbligatori implementati correttamente. È un indicatore critico per il rilascio del prodotto.
Formula	$RObbS = \left(\frac{NRObS}{NROb} \right) * 100$ Legenda: • NRObS: Numero Requisiti_G Obbligatori Soddisfatti • NROb: Numero Totale Requisiti_G Obbligatori
Soglia di Accettabilità	da definire
Soglia di Ottimalità	da definire

Metrica_G	MPD-02 - Requisiti_G Desiderabili Soddisfatti
Descrizione	Indica la percentuale di requisiti _G desiderabili implementati correttamente, aumentando il valore del prodotto senza essere indispensabili al rilascio.
Formula	$RDesS = \left(\frac{NRDS}{NRD} \right) * 100$ Legenda: • NRDS: Numero Requisiti_G Desiderabili Soddisfatti • NRD: Numero Totale Requisiti_G Desiderabili
Soglia di Accettabilità	da definire
Soglia di Ottimalità	da definire

Metrica_G	MPD-03 - Requisiti_G Opzionali Soddisfatti
Descrizione	Indica la percentuale di requisiti _G opzionali implementati, realizzati solo a fronte del completamento del resto del lavoro pianificato.
Formula	$ROpS = \left(\frac{NROpS}{NROp} \right) * 100$ Legenda: • NROpS: Numero Requisiti_G Opzionali Soddisfatti • NROp: Numero Totale Requisiti_G Opzionali
Soglia di Accettabilità	da definire
Soglia di Ottimalità	da definire

5.2.4. Affidabilità

Metrica_G	MPD-04 - Failure Density
Descrizione	Misura la densità di guasti del sistema rispetto alla dimensione del codice sorgente.
Formula	$FD = \frac{NF}{KLOC}$ Legenda: • NF: Numero di Failure rilevate • KLOC: Migliaia di Linee di Codice
Soglia di Accettabilità	da definire
Soglia di Ottimalità	da definire

Metrica_G	MPD-05 - Statement Coverage
Descrizione	Percentuale di istruzioni del codice sorgente coperte dall'esecuzione dei test.
Formula	$SC = \left(\frac{IT}{ITOT} \right) * 100$ Legenda: • IT: Istruzioni Testate dai test automatizzati • ITOT: Istruzioni Totali nel codice
Soglia di Accettabilità	da definire
Soglia di Ottimalità	da definire

Metrica_G	MPD-06 - Branch Coverage
Descrizione	Percentuale di rami del codice che sono coperti da Test automatizzati.
Formula	$BC = \left(\frac{BT}{BTOT} \right) * 100$ Legenda: • BT: Rami (branch) Testati dai test automatizzati • BTOT: Rami (branch) Totali nel codice
Soglia di Accettabilità	da definire
Soglia di Ottimalità	da definire

5.2.5. Usabilità

Metrica_G	MPD-07 - Error Rate
Descrizione	Percentuale di errori rispetto al numero di azioni.
Formula	$ER = \left(\frac{NET}{NAT} \right) * 100$ Legenda: • NET: Numero di Errori Totali • NAT: Numero di Azioni Totali
Soglia di Accettabilità	da definire
Soglia di Ottimalità	da definire

Metrica_G	MPD-08 - Time To Complete Task
Descrizione	Stima il tempo medio necessario a un utente per completare correttamente un'operazione specifica.
Formula	$TCT = \frac{TO + 4 * TP + TPes}{6}$ Legenda: • TO: Tempo Ottimista • TP: Tempo Probabile • TPes: Tempo Pessimista
Soglia di Accettabilità	da definire
Soglia di Ottimalità	da definire

5.2.6. Efficienza

Metrica_G	MPD-09 - Response Time
Descrizione	Tempo impiegato dal sistema per fornire una risposta a seguito di un input dell'utente.
Formula	$RT_{\underline{G}} \leq t$
Soglia di Accettabilità	da definire
Soglia di Ottimalità	da definire

5.2.7. Manutenibilità

Metrica_G	MPD-10 - Coefficient of Coupling
Descrizione	Numero di dipendenza tra i componenti del sistema.
Formula	$COC = \frac{ND}{NC}$ Legenda: • COC: Coefficient of Coupling • ND: Numero di Dipendenze • NC: Numero di Componenti
Soglia di Accettabilità	da definire
Soglia di Ottimalità	da definire

Metrica_G	MPD-11 - Cyclomatic Complexity
Descrizione	Quantifica la complessità logica del codice contando il numero di percorsi indipendenti.
Formula	$v(G) = E - N + 2P$ Legenda: • E: Numero di archi del grafo di controllo • N: Numero di nodi del grafo • P: Numero di componenti connesse
Soglia di Accettabilità	da definire
Soglia di Ottimalità	da definire

Metrica_G	MPD-12 - Instability Index
Descrizione	Misura la resilienza di un modulo al cambiamento. Un valore vicino a 0 indica massima stabilità.
Formula	$I = \frac{C_e}{C_e + C_a}$ Legenda: • C_e: Classi esterne da cui il modulo dipende (Efferent) • C_a: Classi esterne che dipendono dal modulo (Afferent)
Soglia di Accettabilità	da definire
Soglia di Ottimalità	da definire

Metrica_G	MPD-13 - Code Smell
Descrizione	Indica la densità di potenziali problemi di design o violazioni delle best practice di codifica.
Formula	$CS = \frac{NCS}{KLOC}$ Legenda: • NCS: Numero di Code Smell rilevati • KLOC: Migliaia di Linee di Codice
Soglia di Accettabilità	da definire
Soglia di Ottimalità	da definire